

Savings Calculator for Dream House **(Python Program)**

ABSTRACT

This assignment presents a Python program that calculates the number of months required for a person to save enough money for the down payment of their dream house. The program considers the user's annual salary, the percentage of salary they plan to save, and the total cost of the house. In Part B, the program is enhanced by introducing a semi-annual raise, which increases the salary every six months and affects the saving process. The program uses loops, conditional statements, and mathematical formulas to simulate real-life financial planning.

INTRODUCTION

Saving money for a house requires proper financial planning. This program helps users estimate how long it will take to save enough for a down payment based on their salary and savings habits.

The program is divided into two parts:

- **Part A:** Calculates months required without salary increase.
- **Part B:** Calculates months required with a semi-annual salary raise.

The program uses Python basics such as input/output, variables, loops, and conditions.

OBJECTIVES

- To calculate the number of months required to save for a house.
- To apply mathematical formulas for savings and interest.
- To understand the use of loops for repeated calculations.
- To implement conditional statements for salary raise (Part B).
- To simulate real-life saving scenarios using programming.

CONCEPTS USED

- Python variables and data types
- User input using `input()`
- While loop
- Conditional statements (`if`)
- Mathematical calculations
- Financial logic (interest, savings, salary raise)

WORKING OF PART A

In Part A, the program asks the user to enter:

- Annual salary
- Percentage of salary to save
- Total cost of the house

Process

1. The program calculates the **down payment** as 25% of the total house cost.
2. Monthly salary is calculated by dividing annual salary by 12.
3. Each month:
 - User saves a portion of their salary.
 - The current savings earn monthly interest (4% annually).
4. The loop continues until current savings become equal to or greater than the down payment.
5. The total number of months is displayed.

ALGORITHM FOR PART A

Step 1: Start program.

Step 2: Input annual salary, portion saved, and total cost.

Step 3: Calculate down payment = 25% of total cost.

Step 4: Initialize current savings = 0 and months = 0.

Step 5: Calculate monthly salary.

Step 6: Repeat while current savings < down payment:

- Calculate monthly saving.
- Calculate monthly interest.
- Add both to current savings.
- Increment month count.

Step 7: Display number of months.

Step 8: End program.

WORKING OF PART B

Part B includes all features of Part A, with an additional feature:

Semi-annual raise (salary increases every 6 months)

Additional Process

- After every 6 months, the annual salary increases by a given percentage.
- Monthly salary is recalculated after each raise.
- This affects monthly savings and reduces the time needed to reach the goal.

ALGORITHM FOR PART B

Step 1: Start program.

Step 2: Input annual salary, portion saved, total cost, and semi-annual raise.

Step 3: Calculate down payment = 25% of total cost.

Step 4: Initialize current savings = 0 and months = 0.

Step 5: Calculate monthly salary.

Step 6: Repeat while current savings < down payment:

- Calculate monthly saving.
- Calculate monthly interest.
- Update current savings.
- Increment months.
- If months % 6 == 0:
 - Increase annual salary by semi-annual raise.

- Recalculate monthly salary.
Step 7: Display number of months.
Step 8: End program.

DIFFERENCE BETWEEN PART A AND PART B

Part A	Part B
Salary remains constant	Salary increases every 6 months
Simple savings calculation	Dynamic savings calculation
Takes more months	Takes fewer months due to raise

CONCLUSION

This assignment demonstrates how programming can be used for financial planning. Part A shows basic savings calculation, while Part B introduces salary growth, making the program more realistic. The use of loops, conditions, and mathematical formulas helps in understanding real-world applications of Python programming.